

画像解析と音声認識を利用した授業動画の自動チャプタ分割

龍谷大学理工学部数理情報学科

T180022 北村優詩

指導教員 佐野 彰

概要

新型コロナウイルスは私たち学生の生活に様々な悪影響を与えた。今後も新たな変異株の影響から、更なる負担を及ぼすことが懸念される。一方で、コロナ禍ではオンライン授業という画期的な授業形式を生み出された。人が密集せずコロナ禍でも教育が継続できることや移動コストの削減等、対面授業にはないメリットが多く存在する。

本研究では授業動画を自動でチャプタ分割するシステムを作成した。本研究には2つの目的がある。1つは、コロナ禍の学校教育で大量にストックされていく授業動画を用いた復習や閲覧をスムーズに行えるようにすること。もう1つは、コロナ収束後もこれらの授業動画を要所で活用することでオンライン授業の質を高め、推進することである。

本研究では、自動チャプタ分割に画像解析のみを利用したもの、音声認識のみを利用したもの、画像解析と音声認識の両方を利用したものの3種類のシステムをPythonを用いて作成した。画像解析を用いた方法では、OpenCVを活用した平均画素法という画像縮小処理を施し、映像の細かな動きに妨げられない工夫や、動画のシーンに合わせて閾値が変動するような処理を行い、授業動画からチャプタ分割すべきカット位置を検出している。音声認識を用いた方法では、inaSpeechSegmenterという音声区間検出ライブラリで無声区間を抽出し、それぞれの区間の長さが動画の分割点として適切かを判定している。さらに、作成されるチャプタへの情報付与の手段としてSpeechRecognitionとMeCabを利用した音声認識からテキスト情報の抽出を試みた。

結果として、画像解析のみでは過剰な数の検出をしてしまい、誤検出も多かった。音声解析のみでは、検出すべき場所が比較的長い無声区間に含まれることがわかったが、長さというデータだけでは判定が難しく、検出漏れが高頻度で確認された。画像解析でカット検出した位置が動画の分割点として適切かどうかを音声区間検出によって判定する手法では、誤検出の発生頻度に改善がみられた。誤検出や未検出が僅かに発生するため手動で作成するチャプタには劣るが、簡易的なチャプタとしての活用は見込める精度である。一方で、音声認識によるテキスト情報の抽出では、単語の出現数とその単語の重要性にある相関関係が弱く、チャプタへの適切な情報付与が行えなかった。

今後の課題として、自動チャプタ分割では、さらに精度を高めるために重要な検出のみに絞る工夫が必要である。チャプタへの情報付与では、音声認識ではなく、映像からテキスト情報を読み込むなどの異なる手法を取り入れる必要がある。

2021 年度 卒業論文
画像解析と音声認識を利用した授業動画の自動チャプタ分割

龍谷大学 理工学部 数理情報学科
T180022 北村優詩
指導教員 佐野 彰

目次

1	はじめに	1
1.1	研究背景	1
1.2	研究の手順	1
2	画像解析を用いた手法	1
2.1	画像解析を用いた自動チャプタ分割の方法	1
2.2	授業動画のシーンに適応可能な閾値処理	2
2.3	平均画素法を用いた画像処理	2
2.4	画像解析による自動チャプタ分割の考察	2
3	音声認識を用いた手法	3
3.1	SpeechRecognition と MeCab を用いたチャプタへの情報付与	3
3.2	inaSpeechSegmenter を用いた音声区間検出	4
3.3	音声区間検出による自動チャプタ分割の考察	5
4	画像と音声の解析による自動チャプタ分割	5
4.1	処理フロー	5
4.2	実行結果と考察	5
5	おわりに	6
	謝辞	7
	参考文献	8
	付録 ソースコード	9

1 はじめに

1.1 研究背景

新型コロナウイルスは私たちの生活を圧迫し、ひどく苦しめた。令和2年以降、学生の自殺者数は増加傾向にあるという。原因はコロナ禍の影響である可能性が高く、特に「オンライン授業が導入され友人と交流する機会が減ったことや、アルバイトがなくなって経済的に困窮するなど自殺のリスクを高める要因が重なった」と指摘されている。[1] 今後も新たな変異株の発見による影響が学生に負担を及ぼすことが懸念される。

一方で新型コロナウイルスは、学校教育にオンライン授業という画期的な授業形式を生み出した。オンライン授業に切り替えたことで人が密集せず、コロナ禍でも教育の継続が可能である。また、遠方から授業に参加することもできるため、移動コストを削減する機能もある。

本研究では、授業動画を自動でチャプタ分割するシステムの作成に取り組んだ。これを用いて、大量にストックされる授業動画にチャプタを取り付けることで、スムーズに動画を見返すことが可能になり、復習に役立つ。本研究には、オンライン授業をより充実させ、学生側に掛かる負担を軽減する目的がある。また、オンライン授業には対面授業にはないメリットもある。コロナ収束後も要所での活用を推進したい。

1.2 研究の手順

動画は映像と音声の2つの要素からなっている。また、映像は複数枚の静止画像を束ねて構成されている。本研究では、自動チャプタ分割に画像解析のみを利用したもの、音声認識のみを利用したもの、画像解析と音声認識の両方を利用したものの3種類のシステムをPythonを用いて作成した。それぞれの実行結果を比較し、システムごとの特徴などを考察する。

2 画像解析を用いた手法

2.1 画像解析を用いた自動チャプタ分割の方法

チャプタ生成の手法は参考文献 [2] がベースとなっている。動画の中から分割点を判定する基準は時間方向で、隣り合うフレームの画像を比較したときに得られる差の絶対値である。画像の比較を3×3の格子数で色情報を黒と白のみで示したのが図1である。それぞれ黒が0、白が1で構成されており、隣り合うフレームの対応する格子で差を計算している。①では(1 - 0)が3箇所、(0 - 1)が2箇所で行われ、各格子で差を計算している。ここで、全ての格子のもつ差の絶対値をとった後に加算した結果、5という画像全体の差の絶対値が得られている。②は、隣り合うフレームとの比較で差の絶対値が0のとき、つまり映像に動きがない部分である。

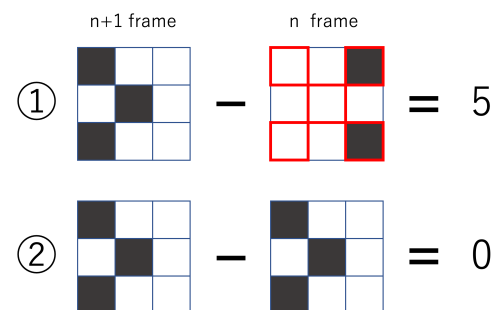


図1 隣り合うフレームでの画像比較

図1のようにして得た差の絶対値を閾値と比較していく。閾値を超えていた場合には、比較した中の後の方のフレームがカット検出として推定される。

2.2 授業動画のシーンに適応可能な閾値処理

授業動画は教員や科目によって多様な形式がある。例えば、PowerPoint のスライドショー、レジュメのスクロール、教員がカメラで映されているものがある。また、中には 1 つの授業動画でこれらの形式が使い分けられているものもある。平均的な差の絶対値はスライドショーやレジュメのスクロールでは小さい。一方で、カメラによる映像では差の大きいフレームが連続することもあり、不安定である。これら全ての映像の処理を 1 つのシステムで可能にするには、随時そのシーンに合わせて閾値を変動させる必要があると考えた。

閾値処理の仕組みを説明する。あるシーンに適応した閾値を求めるために、周辺のフレームから得られる差の大きさの情報を利用している。これにより、そのフレームの前後の映像の様子を推定し、フレームごとに異なる閾値を出力できる。閾値はそのフレームの前後何フレームかがもつ差の絶対値を平均し、平均値を何倍かすることで得られる。本研究では、前後 1 分の映像から得られた差の絶対値の平均値を 20 倍したものを閾値としている。

2.3 平均画素法を用いた画像処理

授業動画からカット検出を行う際、処理を高速化するため、また動画内の物体の細かな動きに検出を妨げられないようにするために、動画から得た画像のピクセル数を少なくする縮小処理を行い、システムの改良を行なった

平均画素法とは画像サイズを縮小する処理方法の 1 つである。縮小処理なので、変換後の画像は変換前よりも総ピクセル数が少なくなる。そのため、変換後の 1 つのピクセルでは変換前の複数のピクセルがもつ色情報を表現する必要がある。

平均画素法の縮小処理の仕組みを図 2 を用いて説明する。平均画素法では、画像を構成するピクセルを四角形で捉える。変換後の赤枠に囲まれたピクセ

ルは、変換前に内にあった A の色情報と跨っている B、C、D の色情報を表現する。そのために、赤枠の中を A、B、C、D のそれぞれが占める面積比を重みとして各色情報に乗算している。それらを加算した結果、赤枠に複数のピクセルの色情報を含んだ色が出力される。この工程を変換後の各ピクセルで繰り返すことで、画像サイズを縮小することが可能である。

本研究では、OpenCV[3] というライブラリの cv2.INTER_AREA を利用して平均画素法を用いた画像処理を行なっている。

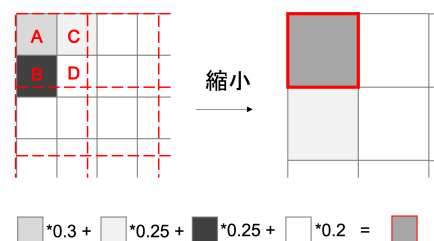


図 2 平均画素法を用いた画像の縮小処理

2.4 画像解析による自動チャプタ分割の考察

ここまで説明してきた画像解析の手法を用いたカット検出を、3 つの授業動画に対して行なった。実行結果を表 1 に示す。検出の評価は私が実際に授業動画を確認したものである。スライドの切り替わるタイミングなどでカット検出が行われていれば、正確な検出と評価する。

表 1 画像解析によるカット検出の評価

	応用プログラミング	クラウドコンピューティング	現象の数理
授業時間	0:56:01	3:09:20	0:18:34
正確な検出	6	55	8
誤検出	5	33	2
未検出	3	6	1

今回の検出元である授業動画は全てスライドショーを軸に構成されたものである。ただし、現象の数理の授業では板書する様子が、クラウドコンピューティングの授業では教員の顔がアップになって話している様子と授業の資料映像が挿入されている。

表1から、全体を通して画像解析を用いたカット検出では、授業時間に対して過剰な数の検出がみられる。この問題は、閾値を引き上げて解消することは難しいと考える。引き上げられた閾値が、重要な検出ごと弾いてしまうからである。全体の検出数から、不必要な検出のみを取り除く工夫が必要だということがわかった。

次に、誤検出が発生するパターンがどの授業動画でも一致していたことがわかった。ライブ形式で行われるオンライン授業のアーカイブでは、通信環境の影響で映像が途切れる箇所が何度も確認できた。そういった状況下で、画面がスクロールされたり、物体が動いたりすると、画面が瞬間的に大きく変わり、誤検出が発生している。また資料映像の挿入が行われることでも、その映像途中で誤検出が発生することがあった。現象の数理の授業動画はこの中で唯一、授業が録画されたものであるために比較的誤検出が少ないと考えられる。

未検出が確認できた箇所では、スライドショーで酷似するデザインのスライドが連続した時に発生している。ただし、未検出数は全体で少なく抑えられており、チャプタ分割に大きく影響しない。

以上の実行結果を通して、過剰な検出数と誤検出が多いことが原因で、画像解析によるカット検出のみを利用した自動チャプタ生成では実際に利用可能なクオリティを満たせない。

3 音声認識を用いた手法

本章では、音声認識を活用するために利用したライブラリ、エンジンについての用途と解説、また、音声認識のみを利用した自動チャプタ分割の実行結果を示す。

3.1 SpeechRecognition と MeCab を用いたチャプタへの情報付与

システムによって分割された区間では、それぞれ何をテーマに授業が行われているのかを調べるために音声認識を利用することを考えた。授業の音声をテキストに起こし、そのテキストを形態素解析によって単語に切り分け、区間ごとに頻出した単語の情報をチャプタに情報として付与することが目的である。

SpeechRecognition[4] とは、Python 上で音声認識を実行可能にするためのライブラリである。このライブラリはいくつかの音声認識エンジンをサポートし、それらを利用することで音声認識を実行している。音声はリアルタイムでマイクに入力するか、用意した wave 形式の音声ファイルとして入力される。しかし、入力可能な音声ファイルの容量には制限があり、授業動画のような長時間の音声は数分ごとに分割して入力する必要がある。

本研究では、比較的精度が高いとされているエンジンである Google Speech Recognition を利用している。また、授業動画の形式は mp4 ファイルに限定し、mp4 ファイルから mp3 ファイルへと音声のみを抽出、さらに mp3 ファイルから wave ファイルへと変換する。この wave ファイルを 3 分ごとに分割して入力し、区切って変換されたものを 1 つにまとめることで動画全体の音声をテキストに起こしている。この手法は参考文献 [5] をベースとした。

MeCab とは、オープンソース形態素解析エンジンである。[6]MeCab に文章を入力すると、図3のように形態素解析の結果を出力できる。出力結果には切り分けられた単語と、その品詞、品詞細分類3つ、活用形2つ、原形、読み、発音が並ぶ。人名や新語、造語については誤検出を起こすことも多いが、一部 MeCab 用の辞書に登録されているものや、登録されていなくても人名や新語、造語であると推定できるものに関しては、「固有名詞」として分類される。

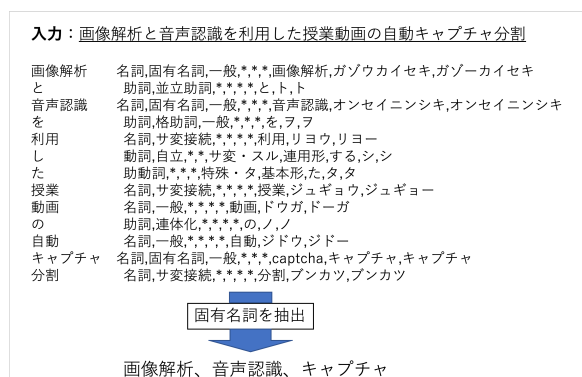


図3 MeCab を用いた形態素解析と固有名詞の抽出

本研究では、MeCab が推奨している IPA 辞書を辞書指定して、授業動画のテキストに対して形態素解析を行ない、名詞のみの抽出を試みた。実行すると、1 時間程度の授業動画からは大量の名詞が抽出され、授業のテーマと重なるような名詞のみの抽出は困難だった。そこで、図 3 のように固有名詞として細分類された単語のみの抽出し集計した。これを、3 つの授業動画に対して実行したものが表 1 である。

表 2 授業動画の頻出単語の一例

応用プログラミング		クラウドコンピューティング演習		現象の数理	
頻出単語	回数	頻出単語	回数	頻出単語	回数
クライアント	48	Microsoft	46	X	9
http	39	クラウド	41	500点	2
ここに	11	AI	33	関係性	2
OS	11	PC	25	波動方程式	2
ゲット	10	Azure	22	2.0	2
Web	9	Windows	17	2020年度	1
1 回	9	アプリ	13	板倉区	1
PC	8	ホロレンズ	11	特化	1
ECHO	6	Xbox	10	無限大	1
HTML	6	スマホ	10	sin	1

表 1 から、「現象の数理」の授業では「応用プログラミング」や「クラウドコンピューティング」の授業よりも頻出する単語に一貫性がなく、単語が現れた回数も非常にまばらであることがわかる。この原因は、実際の授業動画の視聴を通して、「現象の数理」の音質や、単語の発音が比較的良くなかったため、今回の音声認識の手法では単語の認識精度が悪かったためだと考えられる。SpeechRecognition による音声認識で誤検出の多い授業動画では、このような結果が多くみられた。また、どの授業に関しても授業内容に関する重要な単語の抽出にはあまり成

功していない。音声認識が正確に行われた授業動画であっても、単語の現れた回数とその単語の重要性は相関関係にないことが大半だった。したがって、チャプタへの情報付与に音声認識を利用することは適切でないと考える。チャプタへの情報付与という点では、映像から直接テキストを認識して形態素解析を行う手法をとるなど、さらに工夫する必要がある。

3.2 inaSpeechSegmenter を用いた音声区間検出

音声認識を授業動画の自動チャプタ分割に活用するにあたって、教員の話の途切れた箇所、または話していない区間に注目した。授業中、比較的長く誰も話さない区間で話題が切り替わっていると仮定し、無声区間が比較的長い部分を動画の分割点であると判定する。

inaSpeechSegmenter[7] とは、男女の発声区間の割合を測定するために開発された音声区間検出ライブラリである。本来は、男声区間と女声区間を分割する目的で用いられるライブラリである。本研究では、授業動画の音声区間と無声区間を検出し、無声区間の長さの情報を得る目的で利用した。無声区間の長さの測定方法は、まず動画から音声に変換された wave ファイルに inaSpeechSegmenter で音声区間検出を行う。区間はそれぞれ、speech, noenergy, noise, music として検出されるため、speech 以外の区間を全て無声区間として扱う。無声区間の終了時間から、開始時間を引算した結果を無声区間の長さとし、無声区間の終了時間と対応させながら区間ごとにリストに保存していく。作成したリストから無声区間の長い順に、その区間の終了時間を分割点として推定する。

3.3 音声区間検出による自動チャプタ分割の考察

前節で説明してきた音声区間検出手法を用いた自動チャプタ分割を、授業動画に対して行なった。なお、クラウドコンピューティングの授業動画では資料映像の挿入区間が無音であり、本実験の検証結果として適切でないと判断し、応用プログラミングと現象の数理の授業動画のみを対象としている。検出の評価は、その区間が動画の分割点として適切かどうかを私が判断したものである。実行結果を表3に示す。

表3 音声区間検出による自動チャプタ分割の評価

応用プログラミング			現象の数理		
区間終了時間	無声区間(秒)	評価	区間終了時間	無声区間(秒)	評価
0:00:35	32.9	○	0:08:14	11.9	×
0:25:27	8.6	×	0:11:26	8.5	×
0:31:20	10.9	○			
0:38:04	9.5	○			
0:42:21	9.0	○			
0:48:11	9.6	×			

応用プログラミングでは、音声区間検出によるチャプタ分割はそれなりの成功が見られる。一方で現象の数理では、検出した区間の2つがどちらも、分割点として適切ではなかった。比較的長い無声区間が検出されている箇所の多くは、教員の板書などの作業、スライドの切り替わり、資料映像の挿入部分の3つのパターンが多くを占めていた。また、長さの程度であればスライドの切り替わりよりも、教員の作業時間の方が長い区間となる傾向があり、これが原因で、分割点となり得ない区間の検出がそれぞれ2件発生している。

以上の結果から、スライドを切り替えている箇所が無声区間に含まれていることが多く、それが検出可能であることがわかった。しかし本来、分割点として適切な可能性が高いスライドの切り替わりが検出漏れしている場合があるため、スライドの切り替わりの位置を特定する工夫が必要である。したがって、音声認識のみでの自動チャプタ生成は検出位置の精度が低いといえる。

4 画像と音声の解析による自動チャプタ分割

4.1 処理フロー

画像と音声の解析を利用した自動チャプタ分割の処理フローを記す。本研究では、過剰な検出が見られた画像解析によるチャプタ分割を行なった後、音声区間検出を利用して、必要のない分割点を削ぎ落としていく手順で行う。

まず、画像解析によるチャプタ分割を行う。ここで行うチャプタ分割は、2章で説明した手法通りである。画像解析から得た分割点を、最終的なチャプタとして扱う分割点の候補とし、リストに保存する。次に、授業動画の `inaSpeechSegmenter` を用いて音声区間検出を行う。そして、ここで得た無声区間の情報もリスト化する。無声区間の検出を終えた後、画像解析で得た分割点候補のフレームが無声区間に属している場合はその無声区間を、属していない場合はその付近の無声区間の長さで、そのフレームが分割点として適切かどうかを判定する。授業動画から音声区間検出をすると、0秒以下の無声区間が多数検出される。この区間に属しているフレームは動画の分割点から除外していき、1秒以上の無声区間に属している区間を最終的な動画のチャプタとして扱う分割点に決定する。

4.2 実行結果と考察

画像と音声の解析による自動チャプタ分割を3つの授業動画に対して行なった。図4に応用プログラミング、図5にクラウドコンピューティング、図6に現象の数理への実行結果を示す。

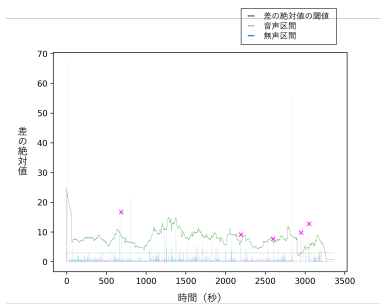


図4 自動チャプタ分割の実行結果 (応用プログラミング)

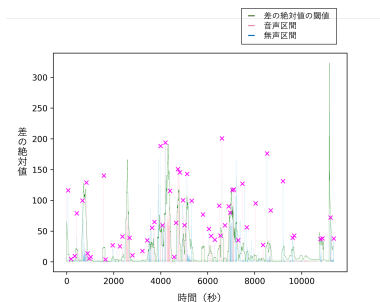


図5 自動チャプタ分割の実行結果 (クラウドコンピューティング)

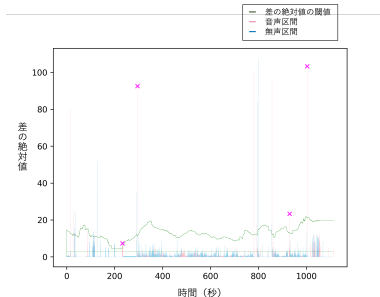


図6 自動チャプタ分割の実行結果 (現象の数理)

また、それぞれの検出結果が適切かどうかを私が確認し、まとめたものを表4に示す。

表4 画像と音声の解析による自動チャプタ分割の評価

	応用プログラミング	クラウドコンピューティング	現象の数理
授業時間	0:56:01	3:09:20	0:18:34
正確な検出	4	48	3
誤検出	1	10	1
未検出	4	2	0

画像解析のみでは過剰であったカット検出の数を、全ての授業動画において約4割程度削減することができた。音声区間の情報から検出を除外する手法が、重要な分割点の除外を防ぎ、未検出数を抑えている。誤検出していた箇所についても、無声区間1秒以上というフィルターによって除外されているものが多く見られる。しかし、応用プログラミングやクラウドコンピューティングでは、誤検出や未検出の発生頻度を減少させたとはいえ複数発生している。これは、ライブ形式の授業なため、録画形式のものに比べ、無声区間が全体的に長いことが原因だと思われる。

5 おわりに

本研究では、実際のオンライン授業でも利用できるようにシステムを作成するために、より精度の高い検出方法を追求してきた。画像と音声の解析を組み合わせることで、誤検出数を抑えられ、システムの改善がみられた。しかし、今回のシステムを利用して授業動画を分割すると、5分から10分おきに1つのチャプタが作成される。これは、多くの授業動画では過剰な分割だろう。また、チャプタの2割程度は誤って自動生成されたものである。したがって、理想的なチャプタ分割はできないため、簡易的なチャプタとしての活用が望ましいと考える。

本研究を経て、今後の課題としてはチャプタ生成位置の精度を高めることと、映像中のテキストを用いたチャプタへの情報付与を可能にすることが挙げられる。

謝辞

本研究活動を進めるにあたり、多くのご指導を頂いた指導教員の佐野さん、並びに同研究室配属の皆さんに、心より感謝致します。

参考文献

- [1] 「国立大生の自殺率が過去 6 年で最多 コロナ禍「困っていれば相談を」」朝日新聞
(参照日:2021/12/28)
<https://www.asahi.com/articles/ASPB13SMFP9WPTIL00Z.html>
- [2] 「Python+OpenCV でアニメのカット検出」(参照日:2021/12/28)
<https://nixeneko.hatenablog.com/entry/2017/09/07/000000>
- [3] 「Home-OpenCV」(参照日:2021/01/08)
<https://opencv.org>
- [4] 「SpeechRecognition」(参照日:2021/01/10)
<https://developer.mozilla.org/ja/docs/Web/API/SpeechRecognition>
- [5] 「python で長い音声・動画からの文字起こしを実装する」(参照日:2021/01/10)
https://www.hamlet-engineer.com/posts/mojiokoshi_long.html
- [6] 「MeCab: Yet Another Part-of-Speech and Morphological Analyzer」(参照日:2021/01/10)
<http://taku910.github.io/mecab/>
- [7] 「ina-foss/inaSpeechSegmenter-GitHub」(参照日:2021/01/10)
<https://github.com/ina-foss/inaSpeechSegmenter>

付録 ソースコード

ソースコード 1 Muviecut.py

```
1 import numpy as np
2 import os
3 import matplotlib.pyplot as plt
4
5 import cv2
6 import shutil
7
8 import subprocess
9 import math
10 import wave
11 import struct
12 from scipy import int16
13
14 # 音声認識
15 import speech_recognition as sr
16
17 # 音声区間検出
18 from inaSpeechSegmenter import Segmenter
19
20 # 形態素解析 MeCab
21 from collections import Counter
22 import MeCab
23 import re
24
25 # メインフォルダ名
26 FORDER = "./py/cut"
27 # 検出元動画フォルダ名
28 MOVIEFILE = "./movie"
29
30 class frametotime:
31     def __init__(self,moviefile):
32         self.cap = cv2.VideoCapture(moviefile)
33         self.fps = self.cap.get(cv2.CAP_PROP_FPS)
34     def __call__(self,frame,diff):
35         sec = frame / self.fps
36         m, s = divmod(sec, 60)
37         h, m = divmod(m, 60)
38         print(f"time({h:.0f}:{m:02.0f}:{int(s):02.0f})\tframe({frame})\tdiff({diff:.2f})")
39
40 class video_data(): # plot 用データ
```

```

41     def __init__(self,filename):
42         self.record = frametotime(filename)
43         self.speech_time = [] # 話し始めのフレームを記録
44         self.noene_time = [] # 話し終わりのフレームを記録
45
46         self.frame_save = [] # フレーム間の数を記録
47         self.thresh_save = [] # 誤差記録、plot 用
48         self.thresh_ave_save = [] # しきい値記録
49         self.cut = [] # カット判定したフレームを記録
50
51 def out_text(video_path, audio_text_path): # mp4 から mp3 に変換
52     out_audio = subprocess.run(
53         [
54             "ffmpeg",
55             "-i",
56             video_path,
57             "-acodec",
58             "libmp3lame",
59             "-ab",
60             "256k",
61             audio_text_path,
62         ]
63     )
64     print(out_audio)
65
66
67 def audio_transcript(audio_change_wav): # mp3 から wav に変換
68     transcript = subprocess.run(
69         [
70             "ffmpeg",
71             "-i",
72             audio_text_path,
73             "-vn",
74             "-ac",
75             "1",
76             "-ar",
77             "44100",
78             "-acodec",
79             "pcm_s16le",
80             "-f",
81             "wav",
82             audio_change_wav,
83         ]
84     )
85     print(transcript)
86

```

```

87 # wav 読み込み-> 音声認識->text 書き起こし
88 def cut_wavtotext(audio_change_wav, cutwav_dir_path, text_data):
89
90     # wav 読み込み
91     wr = wave.open(audio_change_wav, "r")
92
93     # wav 情報を取得
94     ch = wr.getnchannels()
95     width = wr.getsampwidth()
96     fr = wr.getframerate()
97     fn = wr.getnframes()
98     total_time = 1.0 * fn / fr
99     integer = math.floor(total_time)
100     t = 60
101     frames = int(ch * fr * t)
102     # 小数点切り上げ (1 分に満たない最後のシーンを出力するため)
103     num_cut = int(math.ceil(integer / t))
104     data = wr.readframes(wr.getnframes())
105     wr.close()
106
107     X = np.frombuffer(data, dtype=int16)
108
109     # 1 分ごとに wav を分割
110     os.mkdir(cutwav_dir_path)
111     for i in range(num_cut):
112         outf = cutwav_dir_path + "/file" + str(i) + ".wav"
113         start_cut = int(i * frames)
114         end_cut = int(i * frames + frames)
115         print(start_cut)
116         print(end_cut)
117         Y = X[start_cut:end_cut]
118         outd = struct.pack("h" * len(Y), *Y)
119
120         # 書き出し
121         ww = wave.open(outf, "w")
122         ww.setnchannels(ch)
123         ww.setsampwidth(width)
124         ww.setframerate(fr)
125         ww.writeframes(outd)
126         ww.close()
127
128         try:
129             r = sr.Recognizer()
130             with sr.AudioFile(cutwav_dir_path + "/file" + str(i) + ".wav") as source:
131                 audio = r.record(source)

```

```

132         text = r.recognize_google(audio, language="ja-JP").replace(" ", "\n") #
            googlerecognize 使用
133         print(text)
134
135     # 音声認識エラー処理
136     except sr.UnknownValueError:
137         print("Google Speech Recognition could not understand audio")
138     except sr.RequestError as e:
139         print("Could not request results from Google Speech Recognition service;
            {0}".format(e))
140
141     # txt ファイル書き出し
142     open_text = open(text_data, "a", encoding="utf_8")
143     open_text.write(text)
144     open_text.close()
145
146 def speech_segment(obj, audio_change_wav): # 音声区間検出
147
148     # "vad_engine = smn" -> speech, music, noise
149     # "detect_gender = False" -> 性別で分別しない
150     segment = Segmenter(vad_engine="smn", detect_gender=False)
151     # 区間検出
152     seg_data = segment(audio_change_wav)
153
154     if "speech" in seg_data[0]:
155         speech_flag = 0
156     else:
157         speech_flag = 1
158
159     # seg -> 区間, 区間開始(sec), 区間終了(sec)
160     for seg in seg_data:
161         if speech_flag == 0 and seg[0] == "speech":
162             obj.speech_time.append(seg[1])
163             speech_flag = 1
164         elif speech_flag == 1 and seg[0] != "speech":
165             obj.noene_time.append(seg[1])
166             speech_flag = 0
167
168     if seg[0] == "speech":
169         obj.noene_time.append(seg[2])
170     else:
171         obj.speech_time.append(seg[2])
172
173 def slice_text(text_data):
174     with open(text_data) as f:
175         text = f.read()

```

```

176
177 # MeCab 辞書指定:"mecab-ipadic-neologd"
178 mecab = MeCab.Tagger("-d /usr/local/lib/mecab/dic/mecab-ipadic-neologd")
179
180 result = mecab.parse(text)
181 result_lines = result.split("\n")
182
183 result_words = []
184 words = []
185
186 for result_line in result_lines:
187     result_words.append(re.split("[\t,]", result_line))
188
189 for result_word in result_words:
190     if (result_word[0] not in ("EOS", "")) and result_word[2] == "固有名詞":
191         words.append(result_word[0])
192
193 # 頻出単語を降順に 100 個抽出
194 counter = Counter(words)
195 for word, count in counter.most_common(100):
196     print("%s : %s" % (word, count))
197
198 class MovieFrame(object): # フレーム値を返す
199     def __init__(self, moviefile, size=None, inter_method=cv2.INTER_AREA):
200         # 動画ファイルが存在しているか確認
201         self.org = cv2.VideoCapture(moviefile)
202         self.size = size
203         self.inter_method = inter_method
204     def __iter__(self):
205         return self
206     def __next__(self):
207         self.end_flg, self.frame = self.org.read()
208         if not self.end_flg: # 動画が終了したときの例外処理
209             raise StopIteration()
210         return self.frame
211     def __del__(self):
212         self.org.release()
213
214 def MAE(pic): # 絶対誤差
215     return np.mean(np.abs(pic))
216
217 def make_dir(path): # ディレクトリ削除-> 生成
218     if os.path.isdir(path):
219         shutil.rmtree(path)
220     os.makedirs(path)
221

```



```

222 def thresh_check(obj,filename):
223     pic_size = (32, 18) # 縮小後のサイズ
224     frame_cnt = 0 # フレームカウンタ
225     frame_aft = np.zeros((*pic_size[::-1], 3)) # からっぽの画像を生成
226     thresh_size = int(obj.record.fps)*120 # 誤差を記録する配列の大きさ
227     thresh = np.zeros(thresh_size) # 誤差
228     thresh_ave = 0 # 随時平均される誤差の平均
229     cool_cnt = 0 # クールタイムカウンタ
230     cool_time = int(obj.record.fps)*60 # 次のカット判定までのクールタイム
231     cutstart = int(thresh_size/2) # はじめにカット判定が画像処理を待つ時間
232     thresh2 = [] # 山になっている誤差
233     popcount = np.empty(0) # 誤差の期限を数えておく
234
235     for frame in MovieFrame(filename, None):
236         frame_bef = frame_aft
237         # 指定サイズに縮小
238         frame_aft = cv2.resize(frame, pic_size, interpolation=cv2.INTER_AREA)
239
240         # 差分画像作成
241         if frame_cnt != 0:
242             diff = frame_aft.astype(int) - frame_bef.astype(int)
243         else:
244             diff = 0
245         diff_mae = MAE(diff)
246
247         if cutstart <= 0:
248             # しきい値より MAE が大きい場合、カットと判定
249             if thresh[int(thresh_size/2)] > thresh_ave and thresh[int(thresh_size/2)] >
                3:
250                 # クールタイム処理
251                 if cool_cnt >= cool_time:
252                     a = frame_cnt-int(thresh_size/2)+1
253                     obj.record(a,thresh[int(thresh_size/2)])
254                     obj.cut.append(a)
255                     cool_cnt = 0
256                 # 付近で最も差の大きいフレームに更新
257                 elif obj.cut and thresh[int(thresh_size/2)] > thresh[int(thresh_size/2)
                    -cool_cnt]:
258                     a = frame_cnt-int(thresh_size/2)+1
259                     obj.record(a,thresh[int(thresh_size/2)])
260                     obj.cut[-1] = a
261                     cool_cnt = 0
262                 cool_cnt += 1
263                 obj.thresh_save.append(round(thresh[int(thresh_size/2)],2))
264                 obj.thresh_ave_save.append(round(thresh_ave,2))
265         else:

```

```

266         cutstart -= 1
267
268     # 山になっている誤差を抽出してthresh2 に保存
269     if thresh[thresh_size-3] < thresh[thresh_size-2] and thresh[thresh_size-1] <
        thresh[thresh_size-2]:
270         if frame_cnt != 0: # 0 フレーム目を無視
271             thresh2.append(thresh[thresh_size-2])
272             popcount = np.append(popcount,0)
273             if thresh2:
274                 thresh_ave = np.mean(thresh2)*20
275     # thresh2 の更新
276     if popcount.size != 0 and popcount[0] == thresh_size:
277         thresh2.pop(0)
278         popcount = np.delete(popcount,0)
279         if thresh2:
280             thresh_ave = np.mean(thresh2)*20
281
282     thresh = np.delete(thresh,0) # 誤差の記録配列を更新
283     thresh = np.append(thresh,diff_mae)
284
285     frame_cnt += 1
286     popcount += 1
287
288     for f in range(frame_cnt):
289         obj.frame_save.append((f)/obj.record.fps)
290     b = 0
291
292     for p in thresh[int(thresh_size/2):thresh_size-1]:
293         if p > thresh_ave and p > 3: # しきい値より MAE が大きい場合、カットと判定
294             if cool_cnt >= cool_time:
295                 a = frame_cnt-int(thresh_size/2)+1
296                 obj.record(a,p)
297                 obj.cut.append(a)
298                 cool_cnt = 0
299             elif obj.cut and p > thresh[int(thresh_size/2)+b-cool_cnt]:
300                 a = frame_cnt-int(thresh_size/2)+1
301                 obj.record(a,p)
302                 obj.cut[-1] = a
303                 cool_cnt = 0
304
305     obj.thresh_save.append(round(p,2))
306     obj.thresh_ave_save.append(round(thresh_ave,2))
307     b += 1
308     frame_cnt += 1
309     cool_cnt += 1
310     obj.thresh_save.insert(0,0)

```

```

311     obj.thresh_ave_save.insert(0,0)
312
313 def plot(obj,f):
314     print(obj.frame_save[-1],obj.noene_time[-1],obj.speech_time[-1])
315     plt.figure(dpi=350)
316     c=0
317     d=0
318     segall=0
319     cut_seg=[]
320     if obj.noene_time[0]<obj.speech_time[0]: # 無音から始まった場合の plot 処理
321         for k,st in enumerate(obj.speech_time):
322             # カット検出フレーム付近の無声区間を記録
323             if d<len(obj.cut):
324                 # カット検出フレームが音声区間だった場合、前後の無声区間のうち広い方の無声
325                 # 区間を記録
326                 if obj.noene_time[k]>obj.cut[d]/obj.record.fps:
327                     if k==0 or st-obj.noene_time[k]>obj.speech_time[k-1]-obj.noene_time
328                     [k-1]:
329                         cut_seg.append(st-obj.noene_time[k])
330                         segall+=cut_seg[-1]
331                     else:
332                         cut_seg.append(obj.speech_time[k-1]-obj.noene_time[k-1])
333                         segall+=cut_seg[-1]
334                     d+=1
335                 # カット検出フレームが無声区間だった場合、その無声区間を記録
336                 elif st>obj.cut[d]/obj.record.fps:
337                     cut_seg.append(st-obj.noene_time[k])
338                     segall+=cut_seg[-1]
339                     d+=1
340             for i,fs1 in enumerate(obj.frame_save[c:]):
341                 if st<=fs1 or fs1==obj.frame_save[-1]:
342                     plt.plot(obj.frame_save[c:c+i], obj.thresh_save[c:c+i], linewidth =
343                     0.1, color = "red")
344                     c+=i
345                     if len(obj.noene_time)==k+1:
346                         break
347                 for j,fs2 in enumerate(obj.frame_save[c:]):
348                     if obj.noene_time[k+1]<=fs2 or fs2==obj.frame_save[-1]:
349                         plt.plot(obj.frame_save[c:c+j], obj.thresh_save[c:c+j],
350                         linewidth = 0.1, color="blue")
351                         c+=j
352                         break
353             break
354     else: # 音声から始まった場合の plot 処理
355         for k,nt in enumerate(obj.noene_time):
356             # カット検出フレーム付近の無声区間を記録

```

```

353         if d<len(obj.cut):
354             # カット検出フレームが音声区間だった場合、前後の無声区間のうち広い方の無声
               区間を記録
355             if nt>obj.cut[d]/obj.record.fps:
356                 if k==0 or (k+1>len(obj.speech_time) and obj.speech_time[k+1]-nt>
                   obj.speech_time[k]-obj.noene_time[k-1]):
357                     cut_seg.append(obj.speech_time[k+1]-nt)
358                     segall+=cut_seg[-1]
359             else:
360                 cut_seg.append(obj.speech_time[k]-obj.noene_time[k-1])
361                 segall+=cut_seg[-1]
362             d+=1
363             # カット検出フレームが無声区間だった場合、その無声区間を記録
364             elif len(obj.speech_time)!=k+1 and obj.speech_time[k+1]>obj.cut[d]/obj.
               record.fps:
365                 cut_seg.append(obj.speech_time[k+1]-nt)
366                 segall+=cut_seg[-1]
367             d+=1
368         for i,fs1 in enumerate(obj.frame_save[c:]):
369             if nt<=fs1 or fs1==obj.frame_save[-1]:
370                 plt.plot(obj.frame_save[c:c+i], obj.thresh_save[c:c+i], linewidth =
                   0.1, color="blue")
371                 c+=i
372                 if len(obj.speech_time)==k+1:
373                     break
374             for j,fs2 in enumerate(obj.frame_save[c:]):
375                 if obj.speech_time[k+1]<=fs2 or fs2==obj.frame_save[-1]:
376                     plt.plot(obj.frame_save[c:c+j], obj.thresh_save[c:c+j],
                       linewidth = 0.1, color = "red")
377                     c+=j
378                     break
379             break
380         plt.plot(obj.frame_save, obj.thresh_ave_save, linewidth = 0.2, color = "green")
381         plt.plot([0,obj.frame_save[len(obj.frame_save)-1]], [3,3], linestyle="dashed",
           linewidth = 0.2, color = "green")
382         for i,a in enumerate(obj.cut):
383             if cut_seg[i]>=1:
384                 plt.plot(obj.frame_save[a],obj.thresh_save[a],marker = "x",markersize = 5,
                   color = "magenta")
385         plt.savefig(FORDER+"/"+f+"/plot_"+f+".png")
386         plt.clf()
387
388 if __name__ == "__main__":
389     make_dir(FORDER)
390     files = os.listdir(MOVIEFILE)
391     for f in files:

```

```

392     if os.path.isfile(os.path.join(MOVIEFILE, f)):
393         if f == ".DS_Store":
394             continue
395         filename = MOVIEFILE+"/"+f
396         splittextname = os.path.splitext(f)[0]
397         make_dir(FORDER+"/"+splittextname)
398
399         # 分割 wav ファイルの格納フォルダ名
400         cutwav_dir_path = FORDER+"/"+splittextname+"/inwav"
401         # 音声認識テキスト保存ファイル名
402         text_data = FORDER+"/"+splittextname+"/text.txt"
403         # テキスト用音声データ保存ファイル名
404         audio_text_path = FORDER+"/"+splittextname+"/"+splittextname+".mp3"
405         audio_change_wav = FORDER+"/"+splittextname+"/"+splittextname+".wav"
406
407         s = video_data(filename)
408         thresh_check(s,filename)
409         out_text(filename, audio_text_path)
410         # mp3 生成のタイムラグを待つ処理
411         while os.path.isfile(audio_text_path):
412             audio_transcript(audio_change_wav)
413             if os.path.isfile(audio_change_wav):
414                 break
415
416         # 音声認識SpeechRecognition
417         cut_wavtotext(audio_change_wav, cutwav_dir_path, text_data)
418         # 形態素解析
419         slice_text(text_data)
420         # 音声区間検出
421         speech_segment(s,audio_change_wav)
422         # 画像と音声の解析結果から分割点を検出プロット生成,
423         plot(s,splittextname)

```
